Pruebas manuales en Postman

Inicia MySQL y el backend desde la terminal de VS Code. Espera a que Tomcat indique que arrancó en el puerto 8080.

```
.mvnw.cmd spring-boot:run
```

Usa `http://localhost:8080` delante de cada ruta. Puedes crear las peticiones manualmente o importar `docs/AmbiGPS.postman_collection.json`. La colección ya trae los cuatro casos principales y solo requiere configurar su variable `password`.

1. Iniciar sesión y copiar el JWT

Crea **POST /api/auth/login**. En **Authorization** elige **No Auth**. En **Body** → **raw** → **JSON**, pega este cuerpo y escribe la contraseña local de la cuenta.

```
{
  "correo": "usuario1@ambigps.com",
  "password": "REEMPLAZA_CON_TU_CONTRASEÑA"
}
```

Pulsa **Send**. Debe aparecer **200 OK**. En el JSON de respuesta, copia solo el valor de `token`, sin comillas. Para una petición protegida, elige **Authorization** → **Bearer Token** y pégalo en **Token**. Postman agrega el prefijo `Bearer`.

2. Probar los permisos de la guía

Petición	Configuración de Authorization	Esperado
`GET /api/tipos-material`	Bearer Token del usuario	**200** y catálogo
`GET /api/tipos-material`	No Auth. Quita un encabezado Authorization manual si existe.	**401**
`GET /api/roles`	El mismo Bearer Token del usuario	**403**
`GET /api/roles`	Bearer Token de `admin@ambigps.com`. Inicia sesión con esa cuenta primero.	**200** y roles

Como variante del **401**, envía la ruta protegida con el valor `token-invalido` en **Bearer Token**. Para otro contraste de roles, `GET /api/reciclajes` devuelve **403** con el token de `usuario1@ambigps.com` y **200** con el de `reciclador1@ambigps.com`.

Para la sustentación, muestra el método, la ruta y el estado HTTP de cada respuesta. Si importaste la colección, revisa también **Test Results**. Las contraseñas iniciales están en `DataLoaderConfig.java` si todavía usas las cuentas iniciales.

De Postman a MySQL

Estas pruebas escriben registros nuevos en la base local. Usa únicamente los ID que devuelvan tus peticiones para evitar modificar datos existentes.

3. Crear una solicitud como USUARIO

Envía **POST /api/solicitudes** con el token de `usuario1@ambigps.com`. En **Body → raw → JSON**, usa el ID `4` del ciudadano y el ID `3` del material Vidrio, verificados en el esquema local.

```
{
  "usuario": { "idUsuario": 4 },
  "tipoMaterial": { "idTipoMaterial": 3 }
}
```

Debe devolver **201 Created**. Anota el campo `idSolicitud`.

4. Registrar, actualizar y eliminar un reciclaje

Inicia sesión como `reciclador1@ambigps.com`. Envía **POST /api/reciclajes** con ese JWT. Cambia `123` por el `idSolicitud` del paso anterior.

```
{
  "solicitud": { "idSolicitud": 123 },
  "pesoKg": 1.25,
  "descripcion": "Prueba manual de recolección de vidrio"
}
```

Petición	Qué hacer	Esperado
`POST /api/reciclajes`	Anota el `idReciclaje` devuelto.	**201**
`PUT /api/reciclajes/{id}`	Usa ese ID y el mismo JSON con `pesoKg` igual a `2.50`.	**200**
`DELETE /api/reciclajes/{id}`	Elimina solo el reciclaje que acabas de crear.	**204**

5. Verificar los triggers en MySQL Workbench

Abre una pestaña SQL, sustituye `456` por el `idReciclaje` de la prueba y ejecuta:

```
SELECT usuario_actor, accion, fecha_hora,
       datos_anteriores, datos_nuevos
FROM ambigps.auditoria_sistema
WHERE tabla_afectada = 'reciclajes' AND id_registro = 456
ORDER BY id_auditoria;
```

Deben aparecer tres filas con el reciclador como actor. **INSERT** guarda el peso nuevo, **UPDATE** guarda el anterior y el nuevo, y **DELETE** guarda el último valor anterior. Si quieres retirar también la solicitud de prueba, usa `DELETE /api/solicitudes/{id}` con el token de ADMIN y el ID del paso 3.